Appendix of "QTem: Bounded-Error Compressed Data Movement for Temporal Graph Neural Network Training"

Weiwei Ye
The University of Tokyo
Japan
wwye@g.ecc.u-tokyo.ac.jp

Yankai Chen
MBZUAI
United Arab Emirates
yankaichan3@gmail.com

Zhen Wang
Huzhou Normal University
China
zhenwangrs@gmail.com

Dongyuan Li
The University of Tokyo
Japan
lidy@csis.u-tokyo.ac.jp

This appendix accompanies the paper. Sections, tables, figures, equations, and statements that are not part of this document refer to the paper; tables and figures continue its numbering.

A PROOFS

We use the notation of Section 4, where the proofs of Theorem 4.1, Corollary 4.2, and Proposition 4.4 are given.

Proof of Proposition 4.3. Let $\hat{\boldsymbol{\alpha}}$ denote the attention weights computed from the decoded inputs, that is, the softmax of $\hat{\mathbf{z}}$. Then

$$\mathbf{h} - \hat{\mathbf{h}} = \textstyle\sum_n (\alpha_n - \hat{\alpha}_n)\, \mathbf{W}_V \mathbf{x}_n + \sum_n \hat{\alpha}_n\, \mathbf{W}_V (\mathbf{x}_n - \hat{\mathbf{x}}_n).$$

The second sum is a convex combination, so its norm is at most $\|\mathbf{W}_V\|_2\, \varepsilon$. For the first,

$$\Big\| \sum_n (\alpha_n - \hat{\alpha}_n) \mathbf{W}_V \mathbf{x}_n \Big\|_2 \le \|\boldsymbol{\alpha} - \hat{\boldsymbol{\alpha}}\|_1 \max_n \|\mathbf{W}_V \mathbf{x}_n\|_2.$$

The Jacobian of the softmax at any point is $\mathrm{diag}(\boldsymbol{\sigma}) - \boldsymbol{\sigma}\boldsymbol{\sigma}^\top$, and for every $\mathbf{v}$ it satisfies $\sum_i \sigma_i |v_i - \boldsymbol{\sigma}^\top \mathbf{v}| \le 2\|\mathbf{v}\|_\infty$; integrating along the segment from $\mathbf{z}$ to $\hat{\mathbf{z}}$ yields $\|\boldsymbol{\alpha} - \hat{\boldsymbol{\alpha}}\|_1 \le 2\|\mathbf{z} - \hat{\mathbf{z}}\|_\infty$. Finally, $|z_n - \hat{z}_n| \le \|\mathbf{q}\|_2 \|\mathbf{W}_K\|_2\, \varepsilon / \sqrt{d'}$ by the Cauchy–Schwarz inequality. Combining the three bounds proves the claim. If the query is itself computed from a decoded root feature, the same argument adds a term proportional to its input error, again independent of k. □

B CONFIGURATIONS

All models use ETC's architectures [1]: one layer, the 10 most recent temporal neighbors, two attention heads, 100-dimensional hidden states and time encodings, GRU memory for TGN, and a ten-entry mailbox with a transformer memory updater for APAN. Following ETC, the checkpoint with the best validation AP from the fourth epoch on is evaluated on the test split. Table 4 lists the per-configuration settings with which we train every system for the AP of Table 2. Learning rate and dropout on LastFM, the TGAT batch sizes on Wiki-Talk and Stack-Overflow, and the epoch count of GDELT-TGAT differ from ETC's defaults. We fixed them while developing our pipeline, among at most three candidates per configuration and partly on development runs whose test AP we inspected, and then trained these systems with them in fresh runs. The paired comparisons of Section 6.2 are not affected by this choice, because both pipelines of a pair use the same configuration. The epoch times of Table 2 use ETC's defaults throughout and, on Wiki-Talk, 172-dimensional random edge features, the width that ETC lists for this graph [1]; our accuracy runs follow ETC's code, which draws no edge features for Wiki-Talk. Our two pipelines use the same model code; versions of our data path differ only in instrumentation and in options that do not change what a model reads, and the GDELT runs of QTem with seed 7 predate the final version. Every table is encoded by the rule of Section 3.6; GDELT-TGN, which we had first trained with 8-bit codes on blocks of seven entries (97.47 AP with seed 7), enters Table 2 with bit-plane packing (Appendix D).

Table 4: Settings of our accuracy runs (AP in Table 2).

Graph	Model	Batch	LR	Dropout	Epochs	Codec
LastFM	TGN	1000	$2 \cdot 10^{-4}$	0.2	5	8-bit, $b = d$
	TGAT	5000	$2 \cdot 10^{-4}$	0.1	5	8-bit, $b = d$
	APAN	1000	10^{-4}	0.3	5	8-bit, $b = d$
Wiki-Talk	TGN	1500	10^{-4}	0.2	5	8-bit, $b = 4$
	TGAT	1500	10^{-4}	0.1	5	8-bit, $b = 4$
	APAN	1500	10^{-4}	0.1	5	8-bit, $b = 4$
Stack-Overflow	TGN	2000	10^{-4}	0.2	5	8-bit, $b = 4$
	TGAT	2000	10^{-4}	0.1	5	8-bit, $b = 4$
	APAN	2000	10^{-4}	0.1	5	8-bit, $b = 4$
GDELT	TGN	2500	10^{-4}	0.2	5	bit planes
	TGAT	5000	10^{-4}	0.1	10	bit planes
	APAN	2500	10^{-4}	0.1	5	bit planes

C RUNNING THE BASELINES

Common setup. TGL, SIMPLE, ETC, and QTem use ETC's data files, chronological split, and features. The random features of LastFM, Wiki-Talk, and Stack-Overflow are drawn exactly as ETC draws them, from the seed of the run, and GDELT's binary attributes are kept as bytes and converted to 32-bit floats after every gather, which leaves every value a model reads unchanged. To save host memory, the runs of one seed share a read-only copy of the Stack-Overflow tables (45.5 GB), drawn once with ETC's generator and seed, after which the generator state is restored, so that every run continues with the random numbers it would have drawn itself; GDELT runs share one byte-identical copy of the binary table. Each system uses the configurations of Table 4, one negative destination per positive edge drawn uniformly from all nodes, AP averaged over batches, and the checkpoint with the best validation AP from the fourth epoch on, and trains on one A100 GPU of our server. The seeds are 0, 1, and 2, except on GDELT and for SIMPLE on LastFM and Wiki-Talk, where they are 0, 1, and 7.

TGL, ETC, and SIMPLE. We run TGL's released code with a reader for ETC's data files and features. ETC is the full-precision configuration of our code base, in which QTem changes only the representation of static features. SIMPLE builds on TGL's code (Appendix H). All three test a memory-based model after replaying the

training and validation events with the parameters of the selected epoch, as TGL does.

Orca and Zebra. Their AP in Table 2 is the one that ETC reports [1]: ETC trains them with TGN only, with its own configurations on an RTX 3090, and averages three runs; on GDELT, one epoch of either system did not finish within 12 hours. Their epoch times in Table 2 were measured on our server with their released training loops (Orca at commit 2f6d5b4, Zebra at 8e72d65), data loaders that read the same files, and their default TGN models: two graph attention layers over ten neighbors, 100-dimensional memories and embeddings, a learning rate of 10^{-4}, and a dropout of 0.4 (Orca) or 0.3 (Zebra). Edge-feature tables larger than 8 GB stay in host memory and are gathered per batch.

Accuracy of all systems. Table 2 lists the test AP of all systems, and Table 5 the standard deviations of our runs. In ETC's evaluation, Orca and Zebra reach a higher AP with TGN than TGL and ETC on Wiki-Talk and Stack-Overflow, which ETC attributes to their changes to TGN's model structure [1].

D ADDITIONAL ACCURACY RESULTS

Per-seed results. Table 6 lists the paired runs on the benchmark graphs and Table 7 the paired runs on the two real-feature graphs behind Figure 4. Both arms of a pair share the code base, the initialization, the sampler, and the negatives; only the representation of the static features differs. We fixed the equivalence margin of Section 6.2 (±0.25 points) after the paired runs on LastFM and the real-feature graphs and before the first paired run on Wiki-Talk and Stack-Overflow, and did not change it afterwards. Nor do 8-bit codes make training less stable: the seed-to-seed standard deviation of QTem exceeds that of the full-precision pipeline in 6 of the 15 cells, and its median over the cells is 0.20 points against 0.24. Wikipedia and Reddit use TGL's settings for these graphs [3] (batch size 600, learning rate 10^{-4}, ten epochs, dropout 0.2 for TGN and 0.1 otherwise) with ETC's single-layer models, and are evaluated with the same checkpoint rule. The control arm trains the same models with every static feature replaced by zero. It confirms that the LIWC features carry information that the models exploit, most clearly for TGN and TGAT on Reddit, whereas Wikipedia's models depend on them only weakly. On Reddit-APAN, full-precision runs vary by 2.8 points across seeds, so neither the positive difference of QTem nor the negative one of the control is significant there. The 4-bit arm trains the same models with four-bit codes and one scale per row. On both graphs, its mean paired differences to full precision range from −0.03 to +0.50 points, none significant ($p \geq 0.32$); models trained at full precision, by contrast, change their logits by 4–37% when they read such codes (Section 6.3).

Stack-Overflow-APAN. This configuration yields the largest paired differences (−15.9 to −1.1 points, mean −7.16, $p = 0.25$) and is also the least stable one. At full precision, its test AP ranges from 67.9 to 78.7 across seeds although the validation AP of every selected checkpoint lies between 91 and 93, and TGL and SIMPLE, trained with the same configuration, end at 65.66 and 64.16 on average (Table 5). The validation AP of the checkpoints selected by the two pipelines differs by at most 1.5 points. Two controls locate the source of the gap. First, we repeated the full-precision run of seed 0 with identical code, configuration, seed, and inputs: the repetition ends 3.9 points from the original, because nondeterministic memory updates make two runs diverge from the first epoch on. Second, we evaluated the trained models of both pipelines for seeds 0 and 1 with the inputs of either pipeline, following the protocol of Table 2. Re-evaluating a model with its own inputs reproduces the test AP of its training run within 1.10 points, and switching to the inputs of the other pipeline moves it by at most 0.44 points. Reading 8-bit inputs therefore does not lower the AP of a trained model; the gap stems from where training ends, which in this configuration varies by several points even between two identical full-precision runs.

Ranking with 49 negatives. AP with one negative per positive saturates for strong models, so we re-evaluated all 90 paired checkpoints of Tables 6 and 7 on the test split with 49 random negatives per positive, each with the representation it was trained with, and computed the mean reciprocal rank of every positive among its 50 candidates (Table 8). We fixed this analysis before evaluating the Wiki-Talk and Stack-Overflow checkpoints. After Holm's correction for the 15 cells, no cell differs significantly (smallest adjusted $p = 0.12$); the smallest unadjusted value, $p = 0.008$ on Stack-Overflow-TGAT, belongs to a gain of 0.70 points for QTem. Over all 45 pairs, the MRR of QTem differs from full precision by +0.4 points on average (95% CI [−1.5, 2.3]). The nine largest paired differences, 6 to 24 points in magnitude, all fall in the four cells whose full-precision MRR varies most across seeds, by 12 to 21 points: Wiki-Talk-TGN, Reddit-APAN, Wikipedia-TGAT, and Stack-Overflow-APAN. Their sign changes with the seed on Wiki-Talk-TGN and Wikipedia-TGAT, whereas QTem scores higher on every seed of Reddit-APAN and lower on every seed of Stack-Overflow-APAN, the configuration whose AP differs most as well. We repeated the two controls of Stack-Overflow-APAN under this protocol after these pairs were known; they are exploratory and do not enter Table 8. Switching a trained model to the inputs of the other pipeline changes its MRR by at most 0.10 points, whereas the repeated full-precision run of seed 0 reaches 16.2, 3.3 points below the original run and below all three QTem runs (16.5 to 17.0). As for AP, the gap is thus consistent with a difference in where training ends, not with the 8-bit inputs that a trained model reads at test time.

Training dynamics. Final scores could hide differences in convergence, so Figure 9 follows the validation AP of the real-feature runs through all ten epochs. In every panel, the largest gap between the mean curves of the two pipelines, at most 0.54 points, is smaller than the spread across seeds of the full-precision pipeline at the same epoch, and 35 of the 36 runs of both pipelines select their checkpoint from one of the last three epochs.

Earlier single runs. Before the paired runs of Table 6, each configuration on Wiki-Talk, Stack-Overflow, and GDELT had been trained once, and six of them also once at full precision (Table 9). These unpaired runs do not isolate the codec and are superseded by Table 6; we list them for completeness. They show how much memory-based models vary between runs on these graphs: on GDELT-TGN, the lossless bit-plane run (96.40) scored 1.07 points *below* the lossy 8-bit run (97.47) with the same seed, although its inputs are bit-identical to full precision, and the largest gap falls on Stack-Overflow-APAN, the least stable configuration (Table 5).

Table 5: Test AP (%) of our runs in Table 2: mean ± standard deviation over three seeds.

Graph	Model	TGL	SIMPLE†	ETC	QTem
LastFM	TGN	82.33±3.39	81.20±1.69	83.86±0.98	84.07±0.57
	TGAT	69.69±0.49	69.46±0.86	69.52±0.50	69.61±0.48
	APAN	69.47±0.26	67.16±0.12	69.45±0.24	69.20±0.19
Wiki-Talk	TGN	92.76±2.29	89.85±2.46	90.03±3.28	89.94±0.89
	TGAT	86.95±0.09	86.89±0.19	86.85±0.08	86.87±0.07
	APAN	84.75±3.87	83.85±7.06	86.62±0.38	86.43±0.63
Stack-Overflow	TGN	86.55±0.89	85.02±1.93	85.93±0.04	85.83±0.91
	TGAT	87.42±0.25	87.42±0.25	87.44±0.08	87.66±0.05
	APAN	65.66±2.21	64.16±4.19	71.94±5.85	64.78±2.05
GDELT	TGN	98.13±0.26	97.74±0.69	97.84±0.46	97.23±0.89
	TGAT	98.08±0.00	98.08±0.00	98.08±0.01	98.08±0.01
	APAN	96.67±0.28	96.90±0.46	97.48±0.33	97.52±0.35

Table 6: Per-seed test AP (%) on the benchmark graphs. LastFM, Wiki-Talk, and Stack-Overflow: paired runs of the full-precision pipeline and QTem from the same initialization, sampler, and negatives; Δ: mean paired difference with the p-value of a paired t-test. GDELT: bit-plane packing is lossless, so QTem reads exactly the full-precision inputs; its seeds were fixed before any of these runs finished (*: seed 7, from the earlier runs).

		Seed 0		Seed 1		Seed 2*		Mean			
Graph	Model	fp32	QTem	fp32	QTem	fp32	QTem	fp32	QTem	Δ	p
LastFM	TGN	82.78	83.79	84.10	83.69	84.69	84.72	83.86	84.07	+0.21	0.66
	TGAT	69.72	69.62	69.89	70.09	68.94	69.13	69.52	69.61	+0.10	0.44
	APAN	69.44	69.02	69.21	69.40	69.69	69.18	69.45	69.20	−0.25	0.38
Wiki-Talk	TGN	86.25	90.71	91.72	90.15	92.13	88.96	90.03	89.94	−0.10	0.97
	TGAT	86.94	86.87	86.81	86.80	86.80	86.93	86.85	86.87	+0.02	0.81
	APAN	86.21	87.06	86.96	85.80	86.70	86.45	86.62	86.43	−0.19	0.77
Stack-Overflow	TGN	85.93	86.85	85.89	85.53	85.97	85.10	85.93	85.83	−0.10	0.87
	TGAT	87.44	87.66	87.35	87.62	87.52	87.72	87.44	87.66	+0.23	0.01
	APAN	69.22	64.76	78.65	62.73	67.95	66.83	71.94	64.78	−7.16	0.25
GDELT	TGN	98.17		97.11		96.40		97.23		lossless	
	TGAT	98.09		98.07		98.08		98.08		lossless	
	APAN	97.15		97.56		97.84		97.52		lossless	

Table 7: Paired runs on real features (test AP, %; mean ± standard deviation over three seeds). Δ: mean paired difference to full precision, with the p-value of a paired t-test; 4-bit: four-bit codes with one scale per row.

			QTem, 8-bit		4-bit	no features	
Graph	Model	fp32	Δ	p	Δ	Δ	p
Wikipedia	TGN	98.42±0.19	+0.08	0.51	−0.03	+0.03	0.75
	TGAT	89.87±0.01	+0.04	0.71	+0.11	−0.15	0.32
	APAN	94.45±0.36	+0.03	0.93	+0.23	−0.66	0.04
Reddit	TGN	99.57±0.05	+0.05	0.37	+0.01	−0.97	0.007
	TGAT	96.52±0.02	+0.07	0.61	+0.09	−4.53	0.003
	APAN	96.88±1.45	+2.07	0.14	+0.50	−1.36	0.57

E LOCK-STEP REPLAY DETAILS

Table 10 summarizes the LastFM replay of Section 6.3 per model and codec, averaged over the three trained models. The input error is the relative error of the event features that enter messages and mailboxes; the logit deviation and the share of flipped link predictions compare every batch with the deterministic full-precision copy; and the memory deviation is the relative Frobenius distance of the whole memory table, sampled every two batches. The last row of each block compares two copies of the unmodified system. The ratio of memory deviation to input error is 0.14 for TGN and 0.068 for APAN, nearly identical for all three codecs: the memory error is a linear response to the input error with a model-specific gain below one, which the mean-square gains of the trained updaters predict (Appendix F). Maxima of the memory deviation stem from the first batches of the stream, while memories are still close to zero, and from isolated batches in which an APAN mailbox is overwritten; they do not persist, and the median deviation of APAN stays at 0.02–0.04% for both block sizes of the 8-bit codes.

Real features. Figure 10 and Table 11 list the replay of Wikipedia and Reddit behind Section 6.3. Each row averages the three models trained at full precision with different seeds (Section 6.2); deviations are measured against the deterministic full-precision copy over the test segment, and ΔAP is the change of the test AP of the replayed model. With 8-bit codes, no replayed model changes its test AP by more than 0.006 points, and fewer than 0.12% of link predictions

Table 8: Mean reciprocal rank (%) with 49 random negatives per positive on the test split (mean ± standard deviation over three seeds), and the mean paired difference with its 95% confidence interval and paired t-test.

Graph	Model	fp32	QTem	Δ [95% CI]	p
LastFM	TGN	37.83±1.28	38.32±0.59	+0.49 [−2.46, +3.43]	0.55
	TGAT	20.29±0.28	20.42±0.15	+0.13 [−0.28, +0.54]	0.30
	APAN	17.34±0.20	17.01±0.30	−0.33 [−0.99, +0.33]	0.16
Wiki-Talk	TGN	55.90±11.12	57.91±3.32	+2.01 [−33.8, +37.9]	0.83
	TGAT	52.27±0.09	52.35±0.31	+0.08 [−0.60, +0.77]	0.66
	APAN	46.39±0.76	46.16±1.86	−0.22 [−3.25, +2.80]	0.78
Stack-Overflow	TGN	50.03±0.13	48.79±1.84	−1.24 [−5.71, +3.23]	0.35
	TGAT	54.35±0.17	55.05±0.26	+0.70 [+0.43, +0.97]	0.01
	APAN	23.47±6.55	16.68±0.26	−6.79 [−23.4, +9.86]	0.22
Wikipedia	TGN	79.63±2.44	79.50±1.50	−0.13 [−4.55, +4.29]	0.91
	TGAT	59.38±6.87	55.72±6.04	−3.66 [−35.7, +28.4]	0.67
	APAN	58.08±1.56	58.47±4.44	+0.39 [−6.76, +7.54]	0.84
Reddit	TGN	91.27±0.72	91.85±0.78	+0.58 [−2.02, +3.18]	0.44
	TGAT	74.25±0.26	74.33±0.77	+0.08 [−2.46, +2.62]	0.91
	APAN	68.84±10.61	82.40±2.47	+13.6 [−9.95, +37.1]	0.13
All 45 pairs				+0.38 [−1.55, +2.30]	0.70

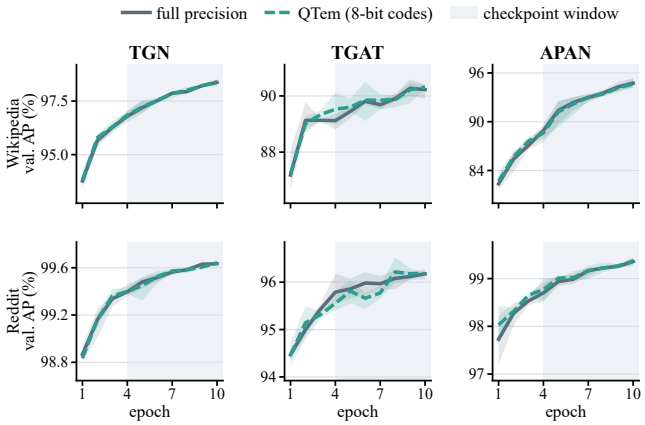


Figure 9: Training dynamics on real features: validation AP after every epoch for the full-precision pipeline and QTem (mean over three seeds; bands: min–max). The shaded region is the window from which the checkpoint is selected.

Table 9: Earlier single unpaired runs (test AP, %), superseded by Table 6; Δ: QTem minus fp32, in points.

Graph	Model	fp32	QTem	Δ
Wiki-Talk	TGN	93.80	92.36	−1.44
Wiki-Talk	APAN	86.56	84.90	−1.66
Stack-Overflow	TGN	86.16	86.02	−0.13
Stack-Overflow	TGAT	87.44	87.26	−0.18
Stack-Overflow	APAN	82.68	71.74	−10.93
GDELT	TGN	98.39	97.47	−0.91

change sign. Destroying the features changes predictions substantially, above all on Reddit-TGAT, whose test AP drops by 23 points with zeros and by 12 with shuffled rows. Four-bit codes are not reliable on real features: on Reddit, their logit deviation ranges from

Table 10: Lock-step replay on LastFM (mean over three trained models; deviations relative, in %).

Model	Variant	Input err.	Logit dev.	Flips	Memory median	Memory max	|ΔAP| (pts)
TGN	QTem, $b = d$	0.65	0.21	0.10	0.09	0.57	≤0.002
	QTem, $b = 4$	0.31	0.10	0.05	0.04	0.27	≤0.001
	4-bit, $b = d$	11.7	3.79	1.80	1.69	10.2	≤0.064
	ETC vs. ETC	0	3.49	1.36	6.03	59.0	≤0.040
APAN	QTem, $b = d$	0.65	0.02	0.00	0.04	1.02	≤0.003
	QTem, $b = 4$	0.31	0.02	0.00	0.02	4.72	≤0.001
	4-bit, $b = d$	11.7	0.41	0.01	0.81	8.68	≤0.018
	ETC vs. ETC	0	0.42	0.01	1.14	18.4	≤0.009

6% to 89% across trained models, and one TGN model loses 5.2 AP points, which is why QTem uses 8-bit codes.

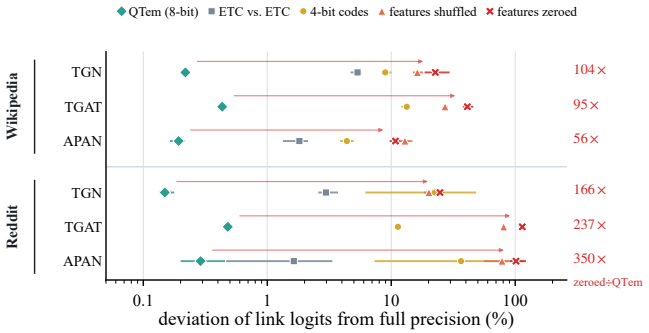


Figure 10: Feature sensitivity of trained models on real features: relative deviation of the link logits from the full-precision copy over the test segment of a lock-step replay (mean over three trained models; bars: min–max) when the models read QTem codes, 4-bit codes, shuffled feature rows, or zeros instead of the original features, and between two runs of the unmodified memory-based models. Right: ratio between zeroed features and QTem.

F CONTRACTION OF TRAINED MEMORY UPDATERS

Section 4 checks the premise of Proposition 4.4 on trained models. We replay the full LastFM stream through the TGN and APAN models trained with seed 0 (Table 6), exactly as ETC evaluates memory-based models, with duplicate memory writes resolved deterministically as in Section 6.3. On every 25th batch, for up to 500 (TGN) or 100 (APAN) positive root nodes, we compute by automatic differentiation the Jacobian of the memory update with respect to every memory block that it reads and to the event features. TGN's GRU reads the node's own memory, as hidden state and as the first part of its message (**A** sums both Jacobians), the other endpoint's memory ($\mathbf{B}_1$), and the event feature (**C**). APAN's transformer updater reads the node's memory (**A**) and the two endpoint memories and the event feature of each of its ten mailbox entries (20 blocks $\mathbf{B}_j$, ten blocks of **C**). On every evaluated batch, our re-implementations

Table 11: Lock-step replay on real features (mean over three trained models). Dev.: relative deviation of the link logits from the full-precision copy over the test segment (%); flips: share of link predictions whose sign changes (%); ΔAP: change of the test AP (points; for QTem the largest magnitude over the three models). ETC vs. ETC compares two copies of the unmodified models; for TGAT, which has no memory, they agree exactly.

		QTem, 8-bit			4-bit codes		shuffled rows		zeros		ETC vs. ETC
Graph	Model	dev.	flips	|ΔAP|	dev.	ΔAP	dev.	ΔAP	dev.	ΔAP	dev.
Wikipedia	TGN	0.22	0.024	0.001	8.9	−0.06	16.2	−0.17	22.7	−0.43	5.35
	TGAT	0.43	0.052	0.005	13.3	−0.26	27.2	−0.83	41.1	−2.88	0
	APAN	0.19	0.011	0.006	4.4	−0.32	12.9	−0.54	10.8	−0.84	1.82
Reddit	TGN	0.15	0.009	<0.001	22.4	−1.75	20.1	−0.35	24.7	−0.37	2.97
	TGAT	0.48	0.112	<0.001	11.3	−0.25	80.7	−12.04	114.0	−23.13	0
	APAN	0.29	0.037	0.004	36.6	+0.90	78.4	−1.48	101.5	−2.49	1.64

reproduce the modules' outputs up to 9.5×10^{-7}. Table 12 summarizes the constants over all sampled updates of nodes whose memory is no longer empty (98% of the samples; first updates start from equal, zero memories in both runs).

Table 12: Jacobian constants of trained memory updaters on LastFM: worst-case gain $\rho = \|\mathbf{A}\|_2 + \sum_j \|\mathbf{B}_j\|_2$ (for TGN the smaller of this and $\sqrt{2}\,\|[\mathbf{A}\,\mathbf{B}_1]\|_2$), own-memory block $\|\mathbf{A}\|_2$, feature gain $L = \sum \|\mathbf{C}\|_2$, mean-square gains g and l (Section 4), and the ratio of relative memory deviation to relative input error predicted by the mean-square steady state and measured in the lock-step replay (Table 10). Median (99th percentile, maximum).

	TGN	APAN
Sampled updates	25,427	5,086
ρ	1.28 (1.86, 2.24)	3.07 (5.49, 20.3)
$\|\mathbf{A}\|_2$	1.03 (1.23, 1.40)	0.83 (1.72, 10.5)
L	0.24 (0.53, 0.66)	0.85 (1.31, 3.53)
g	0.80 (0.82, 0.84)	0.29 (0.45, 1.28)
Share with $g < 1$	100%	99.98%
l	0.049 (0.099, 0.23)	0.052 (0.086, 0.21)
Ratio, predicted	0.11	0.069
Ratio, measured	0.14	0.068

No sampled TGN update satisfies $\rho < 1$; already the block of the node's own memory has $\|\mathbf{A}\|_2 \geq 1.0016$ in every sample, so the worst-case premise fails for TGN. For APAN, the sum over 21 blocks has median 3.07 while a single block is often contractive ($\|\mathbf{A}\|_2 < 1$ in 72% of the samples), so the computable bound cannot certify a contraction. In mean square, both updaters contract: g is at most 0.84 for TGN, and for APAN it exceeds one in a single sampled update. The gains do not drift over the stream; their medians are 0.80, 0.80, and 0.80 for TGN and 0.29, 0.28, and 0.28 for APAN in the training, validation, and test segments. With the root-mean-square values of the event features (1.00 for both models) and of the memories (0.74 for TGN, 0.78 for APAN), the steady state $l\,\sigma_x/\sqrt{1-g^2}$ predicts the ratios in the last rows from the median constants. The prediction assumes uncorrelated, isotropic deviations and is an explanation of the measured behavior, not a bound; it agrees with the replay within a factor of 1.3 for TGN and within 2% for APAN.

G TIMING DETAILS

End-to-end comparison. For Table 2, every run trains with ETC's default configuration on one GPU, without validation, and records the wall-clock time of every training epoch, from its start until the prefetching thread has been joined, including temporal sampling; runs are executed one at a time per GPU. For every configuration, QTem (records in GPU memory, codecs of Table 4) runs directly before and directly after an ETC run on the same GPU, for three, three, and three epochs on LastFM, three, two, and three on Wiki-Talk, and two, one, and one on Stack-Overflow and GDELT. Both systems use the number of host threads of the ETC run listed in Table 2: 32 for LastFM-TGN, one for LastFM-TGAT and LastFM-APAN, and four otherwise. The time of QTem is the mean over all epochs of both of its runs, including the first, and its speedup is the time of the ETC run between them divided by this mean. We fixed these rules before the first run, with one exception: we had planned to divide the ETC time of Table 2 by that of QTem, and after the first configuration we switched to the ETC run of the same session, because the epoch time of ETC varies between sessions. Our ETC runs took 1.05–1.28× as long as the ETC runs listed in Table 2; relative to those, QTem trains an epoch 0.73–1.42× as fast. Peak memory is measured during training only: GPU memory allocated by PyTorch and the resident set of the process, both reset before the first epoch.

Baselines in Table 2. The other systems were timed by the same rules. TGL (commit 241b336) uses eight sampler threads, four intra-op threads (on LastFM, the faster of 4 and 32), and all data in host memory. SIMPLE runs its released code, without the corrections of Appendix H, with fixed batches of the nominal size and the faster of the buffer sizes we timed: one for all data on LastFM and Wiki-Talk and its default of 10% of the data on Stack-Overflow and GDELT; its offline planning is not included. Orca and Zebra train their own TGN models (Appendix C). On Stack-Overflow and GDELT, TGL, Orca, and Zebra train the first 1,000 mini-batches (TGAT: 500) of two epochs, and each epoch is scaled linearly to all of its mini-batches; the setup that TGL's timer records before the first batch is not scaled. ETC's time is the fastest over several sessions and host-thread counts.

Orca and Zebra on TGAT and APAN. Both systems implement TGN only, so Table 2 lists them with TGN. Table 15 gives the epoch times of ports of both to TGAT and APAN, timed by the same

rules; they take 1.1–18.1× the epoch time of QTem. TGAT is the memory-less case of their architecture: the port removes the node memory, uses the node features as layer-0 representations, and keeps their two attention layers (the TGAT of the other systems has one) along with Orca's reuse of embeddings and Zebra's PageRank-based neighbors. APAN keeps ten mails per node, updates memories with TGL's attention-based updater, and delivers every mail to the node and to the neighbors that each system samples. As these models differ from those of the other systems, Table 2 does not list them.

Data-path comparisons. Table 13 lists the epoch time of every timing run behind Section 6.5. In every round, all variants of a configuration trained one epoch back to back on the same GPU, in listed order in odd rounds and in reverse order in even rounds, so that a linear drift over time affects all variants alike. Between rounds, single epochs of a configuration vary by up to 30% for ETC and 57% for uint8+pin, which gather rows on the host, but by at most 19% for QTem-G (Table 13). Before loading its data, every run allocated and released a fixed block of GPU memory, which keeps the memory it needs in its own allocator while it loads; the peak memory in Table 13 is measured after this block was released. SIMPLE ran in the first round only on the three large configurations and in all four rounds on the smaller ones. Speedups are computed per round against ETC in the same round and summarized by their median.

Stack-Overflow-TGN. TGN moves node memories and mailboxes besides static features, and ETC splits the work of a batch between two threads: the prefetching thread constructs the message-flow graphs and gathers, copies, and reconstructs the static rows, 305 s per epoch, while the main thread gathers and copies memories and mailboxes, computes, and updates memories, 610 s per epoch, and waits for the prefetching thread for only 6 s. The static path is thus hidden behind the main thread, yet it is not free: it runs concurrently, and the less static work the prefetching thread performs, the faster the main thread completes its own. Across the five variants, the static gathers, copies, and reconstruction take 162 s per epoch under ETC, 132 s under ETC+pin, 97 s under QTem-H+pin, 45 s under FP16-G, and 47 s under QTem-G, and the main thread's computation, memory updates, and reconstruction take 413 s, 375 s, 367 s, 340 s, and 341 s, respectively (medians over four rounds). Both GPU-resident variants therefore gain alike, QTem-G 1.17× and FP16-G 1.18× over ETC; against FP16-G, QTem gains GPU memory (18.3 GB instead of 24.0 GB).

Smaller graphs. The full-precision stores of LastFM and Wiki-Talk fit on the GPU, so their strongest baseline keeps full-precision tables on the GPU (FP32-G). On these graphs, QTem-G trains an epoch 1.05–1.20× faster than ETC, and an FP32-G epoch takes 0.98–1.01× the time of a QTem-G epoch (medians over four rounds, Table 13); against FP32-G, QTem saves GPU memory, not time.

Loading. Loading includes reading the feature files and, for QTem, the one-time encoding, which takes a median of 52 s on Stack-Overflow and 77 s on GDELT and about a second on the smaller graphs. It adds 43–50 s to the start of a training run on these two graphs, 5–11% of one ETC epoch, once per run.

Earlier series. A first series compared four variants: ETC, ETC with a row-copy gather (ETC+RG), QTem-H without pinned buffers, and QTem-G (Table 14). Single epochs varied by up to 3.0× between rounds there. For TGAT, the results agree with Table 13: QTem-G finished before ETC in every round, with median speedups of 1.54× on GDELT and 1.74× on Stack-Overflow. On Stack-Overflow-TGN, QTem-G was faster as well, with a median speedup of 1.33×, against 1.17× in Table 13; we report the latter. On LastFM, whose epochs take only seconds, all variants were within the run-to-run variation of the series.

H COMPARISON WITH SIMPLE

Setup. SIMPLE [2] decides offline, from the access pattern of one sampled epoch, which rows of each table stay in a GPU buffer during which mini-batches, and updates the buffer between batches; all other rows are gathered from host memory at full precision. We ran its released code with the model configurations of Table 4 and the feature tables of our other runs. The buffer holds 10% of the data, SIMPLE's default, except on GDELT, where it receives the 4.4 GB that QTem-G occupies. On Stack-Overflow, SIMPLE's option to precompute the buffer contents of all batches would need more than 250 GB of host memory, so the buffer contents are derived on the fly from its interval plan, as SIMPLE does when this option is off. The released code trains TGN and TGAT. Its model, mailbox, and planner contain branches for APAN, but its training and evaluation loops lack the steps that TGL performs for APAN; we added them as in TGL, that is, the blocks of the root nodes, the delivery of mails to sampled neighbors, and the mailbox slot of buffered rows, which the release looks up with buffer-local IDs. To run it on our benchmark, we also loaded ETC's feature tables, completed the handling of single-layer models without node memory and the saving of the placement plan, added a seed argument, and fixed several crashes in training and evaluation; none of these changes affects which rows are placed in the buffer, or when.

Correcting the buffer management. Before using SIMPLE, we compared every buffered row with its host copy after each buffer update and found that the released code corrupts buffered data in three ways. (1) It maps buffered rows to buffer slots by their positions in a masked array but compares those positions with the length of the unmasked one, so slots receive other rows' data as soon as a row is evicted. (2) Node memories and mailboxes evicted from the buffer are not written back, so updates made while a node was buffered are lost. (3) When a batch needs fewer rows than the buffer holds, the batch is assembled in a slice of the buffer itself, so the subsequent buffer update and the batch inputs overwrite each other. We corrected all three without changing the placement plan: slots follow the actual buffer layout, evicted state is written back, and batches are assembled in separately allocated memory. Over the first 300 batches of Wiki-Talk-TGN, the released code corrupted all but 79 of 4.0 million buffered node-feature rows, and its per-batch training loss deviated from that of a run keeping all data on the GPU by 48% on average and by up to 130%. After the correction, every buffered feature row matched its host copy on LastFM-TGAT and Wiki-Talk-TGN, and the loss deviated by 0.8% on average and by at most 3.8%, no more than two all-GPU runs deviate from each other because of nondeterministic memory updates (0.7% and 6.2%). All SIMPLE results in this paper use the corrected code.

Results. SIMPLE's epoch times in our data-path comparisons are part of Table 13, and Table 16 breaks them down by SIMPLE's

Table 13: Every timing run of Section 6.5. In every round, all variants of a configuration trained one epoch back to back on the same GPU, in listed order in odd rounds and in reverse order in even rounds. Epoch time of every round, its median (Med.), the median per-round speedup over ETC (Sp.), and the peak GPU memory allocated during the epoch (GB; SIMPLE: its own report). Each configuration lists the variants that apply to its tables: one byte per binary value on GDELT, half precision on Stack-Overflow, and full-precision tables on the GPU on LastFM and Wiki-Talk, whose stores fit there. The runs of LastFM-TGAT and LastFM-TGN come from a later series with the same protocol that also timed TGL and ran SIMPLE in every round; the other configurations come from the first series.

	Epoch time per round (s)						
Variant	1	2	3	4	Med.	Sp.	GPU
GDELT, TGAT							
ETC	621	579	679	643	632	1.00	1.1
ETC+pin	545	575	559	541	552	1.16	1.1
uint8+pin	799	510	526	525	525	1.18	1.1
uint8-G	509	512	521	455	510	1.26	35.9
SIMPLE	712	*first round only*			712	0.87	8.1
QTem-H+pin	560	518	556	466	537	1.17	1.1
QTem-G	491	488	523	461	490	1.28	5.5
Stack-Overflow, TGAT							
ETC	409	354	349	315	351	1.00	0.6
ETC+pin	384	366	362	296	364	1.02	0.6
FP16+pin	349	277	322	268	299	1.17	0.6
FP16-G	261	251	270	259	260	1.35	23.3
SIMPLE	634	*first round only*			634	0.64	9.7
QTem-H+pin	279	435	294	275	287	1.17	0.6
QTem-G	269	312	286	275	281	1.18	17.6
Stack-Overflow, TGN							
ETC	615	739	656	651	653	1.00	1.3
ETC+pin	634	655	635	573	634	1.08	1.3
FP16-G	620	604	570	534	587	1.18	24.0
SIMPLE	1115	*first round only*			1115	0.55	11.4
QTem-H+pin	606	625	556	594	600	1.14	1.3
QTem-G	618	611	522	573	592	1.17	18.3

	Epoch time per round (s)						
Variant	1	2	3	4	Med.	Sp.	GPU
LastFM, TGN							
TGL	36.0	38.3	35.3	37.0	36.5	0.58	0.4
ETC	19.3	21.8	22.8	22.3	22.0	1.00	0.5
ETC+pin	17.1	20.5	21.1	23.9	20.8	1.07	0.5
FP32-G	16.9	18.9	18.0	19.2	18.4	1.16	1.2
SIMPLE	18.6	19.6	18.8	18.0	18.7	1.16	0.4
QTem-H+pin	17.8	21.9	21.1	21.2	21.2	1.07	0.5
QTem-G	15.8	19.6	18.9	18.6	18.8	1.20	0.7
LastFM, TGAT							
TGL	13.7	9.9	10.2	10.9	10.5	0.47	0.6
ETC	6.6	4.6	6.1	4.7	5.4	1.00	0.6
ETC+pin	7.4	5.5	7.4	4.6	6.4	0.86	0.6
FP32-G	5.7	4.0	5.4	4.3	4.9	1.14	1.3
SIMPLE	6.9	4.9	5.5	6.4	6.0	0.95	0.7
QTem-H+pin	6.7	4.3	6.3	4.7	5.5	0.98	0.6
QTem-G	6.0	4.5	5.1	4.1	4.8	1.12	0.8
Wiki-Talk, TGN							
ETC	62.8	70.8	60.8	63.0	62.9	1.00	0.5
ETC+pin	63.4	66.1	60.4	62.4	62.9	1.01	0.5
FP32-G	57.5	59.7	59.3	59.6	59.5	1.07	1.3
SIMPLE	80.1	81.0	88.8	90.7	84.9	0.74	0.7
QTem-H+pin	59.9	63.8	63.1	63.2	63.1	1.02	0.5
QTem-G	57.2	58.5	58.1	64.6	58.3	1.07	0.8
Wiki-Talk, TGAT							
ETC	36.1	34.3	35.4	34.2	34.8	1.00	0.2
ETC+pin	35.7	34.6	33.5	37.5	35.1	1.00	0.2
FP32-G	33.2	32.0	32.4	32.5	32.5	1.08	1.0
SIMPLE	49.8	50.8	51.1	48.2	50.3	0.70	0.4
QTem-H+pin	38.2	34.1	33.4	36.9	35.5	0.98	0.2
QTem-G	33.8	33.6	32.8	33.4	33.5	1.05	0.5

Table 14: Earlier timing series: epoch time in seconds as fastest / median repetition, the speedup of QTem-G over ETC for the fastest / median repetitions, and the median time to load the static features (ETC / QTem-G, including the one-time encoding of QTem).

Graph	Model	Rounds	ETC	ETC+RG	QTem-H	QTem-G	Speedup	Load (s)
LastFM	TGN	2	24.4 / 34.5	22.9 / 24.0	23.9 / 24.0	24.8 / 26.0	0.99 / 1.33	2.1 / 3.0
LastFM	TGAT	2	5.6 / 6.3	6.7 / 9.1	5.7 / 6.7	6.0 / 6.9	0.93 / 0.92	1.9 / 2.3
Wiki-Talk	TGN	2	104 / 105	86.4 / 88.0	86.4 / 94.5	75.2 / 86.0	1.39 / 1.22	2.3 / 2.8
Wiki-Talk	TGAT	2	60.4 / 77.8	66.2 / 67.9	56.1 / 58.1	50.4 / 58.5	1.20 / 1.33	2.8 / 3.4
Stack-Overflow	TGN	4	777 / 954	704 / 922	696 / 834	682 / 717	1.14 / 1.33	131 / 174
Stack-Overflow	TGAT	4	408 / 838	375 / 713	456 / 540	331 / 483	1.23 / 1.74	149 / 195
GDELT	TGAT	4	971 / 1454	748 / 1145	573 / 911	620 / 943	1.56 / 1.54	52.4 / 103

own timers. SIMPLE builds on TGL's code base, so its epoch time also reflects differences in sampling and batch construction; its buffer-management timers isolate the cost of the cache itself. On Stack-Overflow-TGAT, SIMPLE takes 634 s per epoch against 409 s for ETC in the same round. Its buffer reduces the rows gathered from host memory to 64 s of gathering per epoch, but managing the buffer, that is, deriving the buffer contents of each batch from the plan, maintaining flags and indicators, and updating the buffer, costs 183 s, more than ETC's entire feature path. On GDELT-TGAT, with a buffer as large as the QTem records, SIMPLE takes 712 s against 621 s for ETC in the same round: the buffer leaves 159 s of gathering per epoch, and managing it adds another 112 s. On Stack-Overflow-TGN, where the buffer also holds node memories and mailboxes and evicted ones are written back, SIMPLE takes 1115 s against 615 s for ETC, and buffer management costs 386 s per epoch. In each of the three large configurations, the single

Table 15: Epoch time (s) of ports of Orca and Zebra to TGAT and APAN, timed like Table 2, and of QTem (Table 2). *Extrapolated from the first 1,000 mini-batches (TGAT: 500) of two epochs.

	LastFM		Wiki-Talk		Stack-Ov.		GDELT	
	TGAT	APAN	TGAT	APAN	TGAT	APAN	TGAT	APAN
Orca	9.91	14.5	72.1	228.2	1391*	2007*	5095*	1245
Zebra	36.6	41.6	212.4	418.5	2942*	3209*	9838*	4900*
QTem	3.28	13.1	17.7	121.4	175.6	1058	542.5	–

SIMPLE epoch took longer than every one of the four ETC epochs (Table 13), so a single round suffices to rank it. The SIMPLE times in Table 2 come from the released buffer management, without our corrections, which only add work (writing back evicted state and assembling batches in separate memory); they therefore favor SIMPLE. The accuracy of SIMPLE in Table 2 comes from complete training runs with the corrected code, three seeds per configuration (Table 5). On TGAT, which has no memory, the corrected SIMPLE trains exactly like TGL: every SIMPLE run on TGAT that shares its seed with a TGL run reaches the same test AP as that run, namely seeds 0 and 1 on LastFM and Wiki-Talk and all three seeds on Stack-Overflow and GDELT. Its buffer thus delivers exactly the rows that TGL gathers.

Table 16: Where SIMPLE's epoch time goes (seconds, SIMPLE's own timers; medians over the rounds in which SIMPLE ran: the first round on GDELT and Stack-Overflow, all four rounds on the smaller graphs): sampling, construction of message-flow graphs, gathering rows that are not buffered, and buffer management (deriving the buffer contents of each batch from the plan, flags, indicator and buffer updates, and for TGN writing back memories and mailboxes), with SIMPLE's peak GPU memory (GB). ETC: epoch time over the same rounds and its feature path (host gather plus copy, median over all rounds; *: for TGN, the copy step also gathers and copies memories and mailboxes).

		SIMPLE						ETC	
Graph	Model	Epoch	Sample	MFG	Load	Buffer	GPU	Epoch	Feat.
GDELT	TGAT	712	121	44	159	112	8.1	621	239
Stack-Overflow	TGAT	634	58	32	64	183	9.7	409	157
Stack-Overflow	TGN	1115	60	34	180	386	11.4	615	240*
Wiki-Talk	TGAT	50	5.5	3.8	4.3	8.5	0.4	35	5.4
Wiki-Talk	TGN	85	5.5	4.0	12	17	0.7	63	15*
LastFM	TGAT	6.0	1.2	0.5	1.4	0.5	0.7	5.4	1.4
LastFM	TGN	19	1.4	1.2	1.6	3.4	0.4	22	7.8*

I MICROBENCHMARK DETAILS

The microbenchmark of Figure 11, summarized in Section 6.5, fetches R distinct, sorted row IDs drawn uniformly from tables of 4–24 million rows, i.e., far larger than the CPU caches, and synchronizes the GPU after every stage. Each variant is repeated 25 times for $R < 2^{18}$ and 10 times otherwise after three warm-up runs, and we report medians. Table 17 lists the latency at $R = 2^{18}$, including two variants that the figure omits. With pinned staging buffers, which ETC does not use, full-precision fetches become 2.7–3.5× faster, and QTem keeps its lead at equal copy mode: 2.2–3.7× for dense tables and 4.7–6.3× for binary tables. Storing codes and scales in two separate arrays and gathering them with the pipeline's advanced indexing, as a direct port of a row-wise 8-bit quantizer would do, is 1.1–1.5× slower than the fixed-width records at this size, almost entirely because of the host-side gather. Binary tables have no scales; for them, the "sep." column gathers the packed rows with the pipeline's advanced indexing instead of the row copy and thus isolates the gather primitive. At other sizes, the ratio between the two layouts varies widely, from 0.1 to 2.2, because timing noise and fixed per-call costs dominate small fetches. These per-call costs also shrink the advantage of host-resident records for small fetches in general (Figure 11a–c).

Small fetches. At $R = 2^{14}$, close to the per-batch row counts of Stack-Overflow, a fetch from the dense $d = 172$ table with four-entry blocks takes 5.3 ms at full precision (2.6 ms gather, 2.8 ms copy) and 3.9 ms with host-resident records (2.6 ms gather, 1.1 ms copy, 0.2 ms decode). The copy shrinks with the record width, but the gather into a freshly allocated pageable buffer does not, so that host-resident records gain far less than their width suggests (Section 6.5). With a pinned staging buffer, the same fetch takes 0.7 ms, and from GPU-resident records 0.15 ms. A full-precision table that fits on the GPU is read even faster, in 0.06 ms, because it needs no decoding; for the small stores of LastFM and Wiki-Talk, placement at full precision therefore suffices.

Table 17: Fetch latency (ms) at $R = 2^{18}$, median of repeated runs. "pin": pinned staging buffer; "sep.": codes and scales in separate arrays gathered with advanced indexing (binary tables: packed rows gathered with advanced indexing).

	full precision			QTem			
Table	ETC	+RG	+pin	sep.	QTem-H	+pin	QTem-G
dense, $d = 128$	58.7	63.8	20.8	27.9	19.1	9.4	0.91
dense, $d = 172$	81.8	75.5	29.9	32.0	22.5	8.1	1.01
dense, $d = 172$, $b = 4$	81.8	75.5	29.9	42.0	34.2	11.9	1.65
dense, $d = 768$	335.4	354.4	101.7	111.9	96.3	37.1	3.83
binary, $d = 182$	96.4	95.0	32.2	11.9	9.5	6.8	1.60
binary, $d = 413$	209.9	209.2	60.7	14.3	13.0	9.6	2.28

J SCALE RANGE AND BINARY32 SCALES

Theorem 4.1 requires normal binary16 scales. We tested the codec in isolation on adversarial inputs: all-zero blocks, blocks with magnitudes of 10^{-40}, 10^{-6}, 1, 10^{7}, and close to the binary32 maximum, non-uniform blocks, and non-finite values. With binary16 scales, nonzero blocks of magnitude 10^{-6} and below underflow to a zero scale and decode to zero, so that their absolute error equals their magnitude; all-zero blocks decode exactly; and blocks of magnitude 10^{7} and above overflow to an infinite scale and decode to non-finite values. With binary32 scales, the product of a code and a scale is no longer exact in binary32 and adds one rounding, so the argument of Theorem 4.1 yields $|x_{ik} - \hat{x}_{ik}| \le \hat{s}_{ij}(\frac{1}{2} + (2Q + \frac{1}{2})u)$ for every block whose scale is a normal binary32 number, that is, of magnitude at least $Q \cdot 2^{-126} \approx 1.5 \times 10^{-36}$ for 8-bit codes. Smaller blocks get subnormal or zero scales, for which the bound does not hold; for

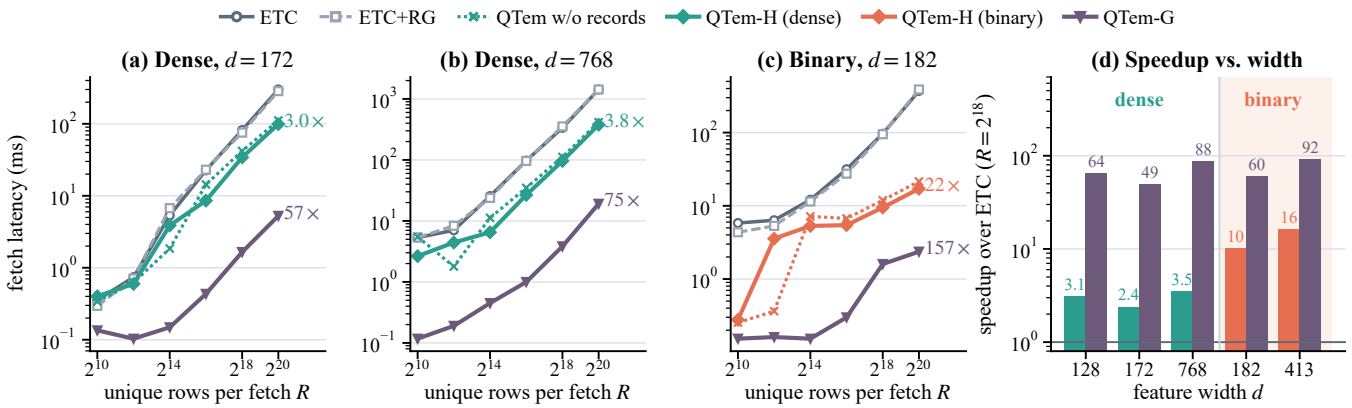


Figure 11: Latency of fetching R deduplicated rows into GPU memory as decoded rows (median of repeated runs). (a)–(c) Latency against R; labels give the speedup over ETC at $R = 2^{20}$. (d) Speedup over ETC at $R = 2^{18}$ per table width.

instance, $\mathrm{fl}_{32}(2^{-149}/Q) = 0$, so such a block decodes to zero with an absolute error equal to its magnitude. The largest error we measured over all regimes with normal binary32 scales is $0.49999\,\hat{s}_{ij}$. Non-finite inputs have no meaningful scale and must be rejected when a table is encoded. Binary32 scales cost two more bytes per block than binary16 scales: with one scale per row, the reduction for $d = 128$, 172, and 768 drops from 3.94, 3.95, and 3.99× to 3.88, 3.91, and 3.98×. We prototyped this variant and tested it on the inputs above, but did not enable it in our experiments: no scale of the two LIWC tables and of one million rows of benchmark-style features leaves the binary16 normal range, only 4 of 192 million four-entry blocks of the MAG sample underflow, and an underflowing block of magnitude below 7.8×10^{-3} incurs an absolute error of at most that magnitude, as Section 4 notes for binary16 scales.

REFERENCES

[1] Shihong Gao, Yiming Li, Yanyan Shen, Yingxia Shao, and Lei Chen. 2024. ETC: Efficient Training of Temporal Graph Neural Networks over Large-Scale Dynamic Graphs. *Proceedings of the VLDB Endowment* 17, 5 (2024), 1060–1072. https://doi.org/10.14778/3641204.3641215

[2] Shihong Gao, Yiming Li, Xin Zhang, Yanyan Shen, Yingxia Shao, and Lei Chen. 2024. SIMPLE: Efficient Temporal Graph Neural Network Training at Scale with Dynamic Data Placement. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–25. https://doi.org/10.1145/3654977

[3] Hongkuan Zhou, Da Zheng, Israt Nisa, Vasileios Ioannidis, Xiang Song, and George Karypis. 2022. TGL: A General Framework for Temporal GNN Training on Billion-Scale Graphs. *Proceedings of the VLDB Endowment* 15, 8 (2022), 1572–1580. https://doi.org/10.14778/3529337.3529342